Aurum Market - Búsqueda semántica, operaciones sobre la colección y control de altas duplicadas

Informe de la práctica de Bases de Datos Vectoriales - Diego Gil Sánchez

Resumen: el sistema ingiere el catálogo completo de Aurum Market (15000 productos) en una base de datos vectorial persistente (Qdrant (servidor), colección 'aurum_catalogo'), con embeddings del modelo intfloat/multilingual-e5-small y la composición de texto 'c3_completo'. Expone búsqueda semántica global, búsqueda filtrada por marca ejecutada en el motor, mutaciones idempotentes del catálogo y una regla calibrada de detección de duplicados. Toda la evidencia es reproducible desde notebook/Aurum_Market_Practica.ipynb.

1. Problema y contrato del sistema

Aurum Market necesita que un cliente describa su intención en lenguaje natural y reciba productos relevantes del catálogo; que las búsquedas por marca devuelvan solo esa marca; que las revisiones y bajas de fichas sean visibles en el buscador sin reindexados manuales; y que las altas duplicadas se detecten antes de ensuciar el catálogo. El contrato medible: top-10 válido para las 12 consultas ciegas; 4/4 consultas filtradas con el 100% de la marca pedida; 24 eventos aplicados, verificados e idempotentes; decisión y candidato para las 14 altas ciegas; latencia y persistencia medidas.

2. Arquitectura

CSV del catálogo -> preparación de texto (sin nulos, descripción truncada a 600 caracteres) -> embeddings normalizados (384 dims) -> Qdrant en Docker con volumen persistente. Colección con distancia coseno e índice HNSW explícito (m=16, ef_construct=128, ef de búsqueda=128). Payload por punto: product_id, title, brand, brand_norm (normalizada para filtrar), color, locale, catalog_version y active, con índices keyword/bool. Los IDs de punto son los UUID estables record_id, lo que hace la reingesta idempotente.

3. Metodología y comparaciones controladas

- Protocolo fijo: mismas 8 consultas de desarrollo, mismos 248 juicios ESCI, k=10, métricas corregidas (IDCG con todos los juzgados; recall/MRR solo sobre relevantes).
- Baseline léxico TF-IDF sobre el mismo corpus y la misma composición de texto.
- Selección de modelo con búsqueda exacta (aísla la representación del índice ANN).
- Composiciones de texto comparadas cambiando un único factor a la vez.
- Fidelidad ANN medida contra oráculo exacto del propio motor (exact=True).
- Latencia: 3 repeticiones de calentamiento + 20 medidas x 8 consultas; percentiles reales con np.percentile.

4. Resultados

4.1 Comparación de modelos (muestra de desarrollo, búsqueda exacta):

Modelo	nDCG@10	Recall@10	MRR@10
sentence-transformers/all-MiniLM-L6-v2	0.482	0.274	0.781
intfloat/multilingual-e5-small	0.715	0.316	0.906

4.2 Composiciones de texto (modelo ganador):

Composición	nDCG@10	Recall@10	MRR@10
c1_titulo	0.602	0.290	0.775
c2_titulo_atributos	0.699	0.350	0.844
c3_completo	0.715	0.316	0.906

4.3 Sistema final frente a baseline (catálogo completo, 15000 productos):

Sistema	nDCG@10	Recall@10	MRR@10

TF-IDF (baseline léxico)	0.403	0.171	0.766
Denso (multilingual-e5-small + Qdrant)	0.562	0.221	0.750

4.4 Fidelidad del índice HNSW frente al oráculo exacto (20 consultas):

hnsw_ef	recall@10 medio	recall@10 peor consulta	p50 (ms)	p95 (ms)
4	1.0000	1.0000	14.73	25.13
16	1.0000	1.0000	15.45	26.82
64	1.0000	1.0000	15.36	27.16
128	1.0000	1.0000	15.53	26.82
256	1.0000	1.0000	5.25	26.84

4.5 Latencia extremo a extremo (consulta en texto -> top-10):

Etapa	p50 (ms)	p95 (ms)	muestras
Codificación de la consulta	9.77	13.25	160
Búsqueda en el motor	6.46	24.15	160
Total	19.49	33.69	160

4.6 Duplicados: regla calibrada en desarrollo con candidatos de la base vectorial: score_top1 >= 0.9093 y margen (top1-top2) >= 0.0 -> precision=1.0, recall=1.0, F1=1.0. La regla congelada se aplicó a las 14 altas ciegas (results/resultados_duplicados.csv).

5. Atribución de errores (resumen; evidencia en el notebook, sección 14)

- Representación: en la peor consulta de desarrollo, el top-10 del ANN coincide con el exacto, luego la pérdida es del modelo/texto (p. ej. atributos como talla o negaciones que el embedding no captura), no del índice. Lo corrige un modelo mayor o un reranker.
- Índice ANN: con ef muy bajo se pierden vecinos del oráculo; se recupera subiendo hnsw_ef (tabla 4.4). Es un compromiso fidelidad-latencia controlado y medido.
- Datos/filtros: 66 marcas del catálogo tienen varias grafías; un filtro de igualdad exacta sobre 'brand' pierde resultados. Se corrige en la capa de ingesta con 'brand_norm' (normalizada) y filtrando sobre ella en el motor.
- Regla de duplicados: los scores de las altas casi-duplicadas se comprimen cerca del umbral; con 14 casos de calibración hay riesgo de sobreajuste. Mitigación: más etiquetas y coste asimétrico explícito (un falso positivo fusiona fichas distintas).

6. Limitaciones

- 8 consultas de desarrollo son pocas para diferencias finas entre configuraciones.
- Los juicios ESCI no cubren todo el corpus: los no juzgados cuentan como relevancia 0, por lo que las métricas son una cota inferior.
- Latencias medidas en un único equipo local sin red (entorno registrado en el JSON).

7. Recomendación de negocio

Desplegar el motor denso multilingüe con hnsw_ef=128 y el filtro de marca normalizado. La evolución con mayor retorno es un reranker cross-encoder sobre el top-50 del motor (ataca la capa dominante de error, la representación) y ampliar los juicios de desarrollo para calibrar sin sobreajuste. Propuestas futuras no implementadas: búsqueda híbrida (denso + BM25 con fusión RRF), cuantización escalar para reducir memoria y monitorización de deriva del catálogo.

Anexo: entorno de medición

Python 3.14.3 - Windows-11-10.0.26200-SP0 - 16 núcleos. Paquetes: sentence-transformers 5.7.0, qdrant-client 1.20.0, torch 2.15.0, numpy 2.5.4, pandas 3.1.0, scikit-learn 1.8.0.